Ders 7 Genişletilmiş Notu: Ara Sınav Öncesi Tekrar: Proje Yönetimi Yol Haritası ve Proje Planı

Ders 7 Genişletilmiş Notu: Ara Sınav Öncesi Tekrar: Proje Yönetimi Yol Haritası ve Proje Planı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Tekrar Dersinin Çerçevesi
- 2. Yazılım Proje Yönetiminin Üç Bilgi Alanı
- 3. Dersin Kaynakları
- 4. Proje Yönetimi Yol Haritası
- 5. Proje Planında Neler Olmalı?
- 6. İş Kırılım Yapısı: Temel Fikir
- 7. Zaman Çizelgesi ve Aktiviteler Arası İlişkiler
- 8. İş Kırılım Yapısı Örnekleri: Şelale ve Sarmal Model
- 9. Planın Devamı: İş Gücü, Görev Ataması, Bütçe ve Kalite Planı
- 10. Yazılım Gereksinim Spesifikasyonu
- 11. Ekip ve Organizasyon Yapıları
- 12. Proje Zamanlaması: Neden CPM Yerine PERT?
- 13. Yazılım Ölçümü
- 14. Maliyet Tahmin Yöntemleri
- 15. Risk Analizi ve Önlemler
- 16. Planın Çalıştırılması: Hazırlık, Planlama ve İzleme

1. Tekrar Dersinin Çerçevesi

Hoca derse, bu hafta önce genel konuların tekrarını yapacağını söyleyerek başladı. Tekrarın omurgası iki şeydi: proje yönetimi yol haritası ve proje planının maddeleri. Hoca bunlara ek olarak fizibilite analizine, özellikle ekonomik fizibilitedeki geri ödeme (payback) analizine ve bilgi sistemleri projelerindeki proje yönetimi aşamalarına da değindi. Bunların temel bilgileri ders dışında paylaşılan slaytlarda yer alacaktı.

Hoca dersin sonuna doğru payback analizine ve ekonomik fizibilite tanımlarına dönüp bir hatırlatma yapmak istediğini söyledi. Ne var ki bu hatırlatmanın içeriği kayda geçmedi.

2. Yazılım Proje Yönetiminin Üç Bilgi Alanı

Hoca ilk dersteki bir slayda döndü. Slaytta üç halka vardı ve her halka yazılım proje yönetimi için gereken bir bilgi birikimini gösteriyordu: proje yönetimi bilgi ve deneyimi, genel yönetim bilimi ve yazılım mühendisliği teknikleri.

Proje yönetimi bilgi ve deneyimi. Bu alan fizibilite çalışması yapmayı, kaynakları ve zamanı yönetmeyi, proje adımlarını genel proje yönetimi çerçevesinde takip etmeyi kapsar. Hoca burada alt yüklenicileri ve dış kaynak kullanımıyla (outsource) yaptırılan uygulama parçalarını örnek verdi. Bunların nasıl takip edileceğini ve bu işte geçerli kuralları bilmek gerekir.

Genel yönetim bilimi. Hoca geçen derslerde üzerinde durulan organizasyon yapılarını hatırlattı. İşletme bilimindeki organizasyon konusuna hâkim olmak gerekir. Bu bilgiye iki nedenle ihtiyaç vardır:

- Bir kurumun içinde proje geliştirme birimi olarak çalışılır.
- Bir ekip yönetilir.

Yazılım mühendisliği teknikleri. Hoca bu halkanın da eksik olmaması gerektiğini söyledi. Proje yöneticisi yazılım mühendisliği tekniklerini takip edebilir durumda olmalıdır.

> **Hocanın vurgusu:** Yazılım proje yönetiminde üç tarafta da bilgi sahibi olan biri proje yöneticisi olarak başarılı olabilir. Özellikle proje yönetimi yol haritasını takip edebilir durumda olmak gerekir.

3. Dersin Kaynakları

Hoca, bilgi sistemleri proje yönetimi (information systems project management) üzerine çok sayıda kitap olduğunu, bunlardan herhangi birine bakılabileceğini söyledi. Kendi ders notlarını hazırlarken kullandığı kaynaklar ise şunlardır:

- Jolyon Hallows, *Information Systems Project Management*, AMACOM yayınları. Proje yönetimi yol haritası (project management roadmap) kavramını hoca bu kitaptan aldı. Kitabın temel hatları internette bulunabilir, eski baskılarına daha kolay ulaşılabilir.
- Pressman'ın yazılım mühendisliği kitabının proje yönetimiyle ilgili bölümleri.

Yol haritası yalnız bu kitaba özgü bir kavram değildir. Project Management Institute (PMI) tarafından hazırlanan web sayfasında da bir projenin nasıl yönetileceği ve proje yönetimi yol haritasının ne olduğu anlatılır.

> **Ders sonrası not:** Altyazıda yazar "Jolion Halos/Halovsun", yayınevi "Amakom", diğer yazar "Presma" olarak geçiyor. Kastedilenler büyük olasılıkla Jolyon Hallows, AMACOM ve Pressman'dır.

4. Proje Yönetimi Yol Haritası

Hoca kavramın ana fikrini bir uyarıyla verdi: Proje yönetimi yalnızca plandan ibaret değildir. "Planı hazırladık, proje yönetimini gerçekleştiriyoruz" demek doğru olmaz. Yol haritası plandan önce gelen bir giriş ve hazırlık aşamasıyla başlar, planla sürer, planın çalıştırılmasıyla devam eder.

Hallows'un kitabı hazırlık aşamasını ikiye böler. Böylece yol haritası dört adımdan oluşur:

1. Projenin anlaşılması (understanding the project).
2. Projenin tanımlanması (defining the project).
3. Proje planının oluşturulması.
4. Planın çalıştırılması (running) ve takip edilmesi.

4.1 Projenin Anlaşılması

Bu adımda projenin başlangıç noktası ve niteliği anlaşılmaya çalışılır. Hocanın sıraladığı sorular şunlardır:

- Bu proje nereden geldi, kim başlattı?
- Bu projenin başlangıcı olan bitmiş başka bir proje var mı? Devam eden ya da bitmek üzere olan bir proje var mı?
- Proje ne tiptir? Sıfırdan yeni başlayan bir proje mi, başka bir yazılımın iyileştirilmesi mi, yoksa yalnızca bir fizibilite analizi projesi mi?
- Projenin sahibi kim, hangi departman? Bu departmanda en son yapılan proje hangisiydi?
- Projenin hedefleri ve öncelikleri neler?

Son soru planı doğrudan etkiler. Bir projenin önceliklerinden söz edildiğinde proje başarı faktörlerine bakılır. Başarı faktörleri projenin zamanında bitmesi, istenen bütçeyle bitmesi ve kapsamının uygun olmasıdır. Projenin sahibi olan departmanın bunlardan hangisine öncelik verdiği proje anlaşılırken öğrenilmelidir, çünkü plan buna göre oluşturulacaktır.

4.2 Projenin Tanımlanması

Tanımlama aşamasında iki şey kayıt altına alınır: projenin çıktıları ve değişikliklerin nasıl takip edileceği.

Çıktılar için hoca bir dizi soru sıraladı. Proje sonunda kullanıcılara ne verilecek?

- Kod verilecek mi?
- Gereksinim analizi sırasında üretilen yazılım gereksinim spesifikasyonu (SRS, software requirements specification) belgesi verilecek mi? Bu belgede kullanım senaryosu (use case) diyagramları olacak mı?
- Proje planı aşamasında plan dokümanı verilecek mi?
- Tasarım sırasında hangi belgeler verilecek?
- Test verileri aktarılacak mı?
- Yazılımın nasıl kullanılacağını anlatan kullanıcı belgeleri verilecek mi?

İkinci konu, yapılandırma yönetimi (configuration management) dediğimiz iştir: Projenin her aşamasında yapılan değişiklikler nasıl takip edilecek? Hocanın örneği şöyleydi: Plan hazırlandı, proje çalıştırılmaya başlandı. Sonra proje sahibi kapsamda bazı değişiklikler istedi ya da başka çıktılar da elde etmek istedi. Bu değişiklik nasıl takip edilecek, plana nasıl aktarılacak? Planda ve bütçede değişiklik gerekiyorsa bu nasıl belirlenecek? Hoca bunların proje planlanmadan önce tanımlanması gerektiğini söyledi.

4.3 PMI Yaklaşımıyla Fark

PMI bu iki adımı anlaşılma ve tanımlanma diye ayırmaz. Bu kısım proje giriş aşaması ya da proje hazırlık aşaması olarak da adlandırılabilir. Hallows'un ayrımı önce projenin niteliğini, sahibini ve hedeflerini, sonra çıktılarını ve değişiklik takibini ele alır.

Hazırlıktan sonra proje planı oluşturulur, plan oluşturulduktan sonra da çalıştırılır.

> **Hocanın vurgusu:** Proje yol haritası dendiğinde proje yalnızca planla bitmez. Plandan önceki hazırlık aşamaları, plan, planın çalıştırılması ve takip edilmesi hep birlikte proje yönetimini oluşturur.

5. Proje Planında Neler Olmalı?

Hoca, proje planında mutlaka olması gerekenleri öğrencilerden saymalarını istedi.

Soru: Bir proje planında neler olmalı?

Cevap (bir öğrenci): Önce bir yol haritası çiziliyordu. Projenin başlangıç tarihi, kaç kişinin çalışacağı, süresi, bitiş tarihi, yapılması gerekenler ve fizibilite analizleri yer alır. Hem ekonomik ve mali açıdan fizibilite analizi hem de risk analizleri olmalıdır.

Hoca bu cevaptan sonra proje planı slaytını açıp maddeleri tek tek geçti. Planın başında projenin kapsamı gelir: projenin hedefi ve sınırları. Ardından tanımlama aşamasında karar verilen yazılım geliştirme süreç modeli yazılır. Sonra risk analizi gelir, yani proje çalışılırken hangi risklerin bulunduğu.

Hoca slaytta olmayan bir maddeyi de ekledi: projenin varsayımları ve kısıtları (assumptions and constraints). Burada hem proje süreci hem de projenin kapsamı açısından nelerin var olduğu varsayıldığı ve hangi kısıtların bulunduğu belirtilir. Varsayım ve kısıtlar, risk analizinde üzerinde durulan kategorilere benzer kategorilerle tanımlanabilir.

Dersin ilerleyen bölümlerinde görev atama, bütçe ve ekip yapısı da eklenince planın maddeleri şu hâli aldı:

1. Projenin kapsamı: hedefi ve sınırları.
2. Yazılım geliştirme süreç modeli.
3. Risk analizi.
4. Varsayımlar ve kısıtlar.
5. Alt aktiviteler: iş kırılım yapısı (WBS, work breakdown structure).
6. Zaman çizelgesi ve aktiviteler arası ilişkiler.
7. İş gücü.

8. Görev ataması ve proje ekip yapısının belirlenmesi.
9. Bütçe.
10. Proje kalite planı.
11. Yazılım gereksinim spesifikasyonu.

Hoca, bir proje planı hazırlarken bu maddelerin ya da IEEE standartları içindeki proje planı standardının kullanılabileceğini, ikisinin de mümkün olduğunu söyledi.

6. İş Kırılım Yapısı: Temel Fikir

Planın alt aktiviteler maddesi iş kırılım yapısıdır. Hoca bunu, kullanılan yazılım geliştirme modelinden yola çıkarak hatırlattı.

Soru: Şelale (waterfall) modele göre bir yazılım geliştirme projesinde hangi alt işlemler vardır?

Cevap (bir öğrenci): Dört temel işlev vardır: ön inceleme ve analiz, tasarım, gerçekleme, son olarak test ve bakım.

Hoca bu adımların WBS'in temel aktiviteleri olarak 1.0, 2.0, 3.0 ve 4.0 biçiminde tanımlanabileceğini söyledi. Bu birinci düzeydir. İkinci düzey ayrıntıya geçilecekse 1.0 başlığındaki "ön inceleme ve gereksinim analizi" şöyle açılabilir:

```
1.0 Ön inceleme ve gereksinim analizi
    1.1 Ön inceleme
    1.2 Gereksinimlerin ayrıntılarını belirleme, bilgi toplama
    1.3 Modelleme
    1.4 SRS belgesinin yazılması
```

Daha fazla ayrıntı istenirse üçüncü düzeye geçilir. Hoca örnek olarak yalnızca 1.3 modellemeyi açtı:

```
1.3 Modelleme
    1.3.1 Use case diyagramının oluşturulması
    1.3.2 Use case belgesinin oluşturulması
    1.3.3 Kavramsal sınıf diyagramının oluşturulması
```

WBS iki biçimde gösterilebilir. İlki, 1.0, 1.1, 1.2 gibi her alt düzeyin içeriye doğru aynı hizaya geldiği hiyerarşik numaralandırmadır. İkincisi ağaç diyagramıdır: Genel projenin dört alt aktivitesi varsa bunlar bir ağacın dalları olarak çizilir. Her iki gösterimde de amaç aynıdır: Bu proje hangi alt işlemlerle tamamlanabilir, bunun planını vermek.

7. Zaman Çizelgesi ve Aktiviteler Arası İlişkiler

WBS'ten sonra zaman çizelgesi oluşturulur. Alt aktivitelerin her birinin ne kadar süreceği, daha önceki çalışmalardan gelen bilgi birikimine göre belirlenir. Ön inceleme ne kadar sürecek, gereksinim analizi ne kadar sürecek, alt işlemler bu projenin büyüklüğüne göre önceki projelere kıyasla nasıl değişir? Zaman çizelgesi oluşturulurken aktiviteler arasındaki ilişkiler de ortaya çıkarılır.

Hoca bu noktada öğrencilere ilişki tiplerini sordu ve soru birkaç adımda netleşti.

Soru: Analiz aşamasının alt aktiviteleri bilgi toplama, modelleme ve SRS belgesinin oluşturulmasıdır. Bu aktiviteler arasında nasıl bir ilişki vardır?

Cevap (bir öğrenci): Biri bittiğinde diğeri başlayabilir. Birbirine bağımlı aktiviteler de olabilir. Bağımsız ilerleyenler de, biri bitmeden diğerine başlanamayanlar da olabilir.

Hoca bu cevabı doğru buldu, ama bunun bir usulü ve belli adları olduğunu hatırlattı. Başka öğrenciler ilişki tiplerini saydı: bitişten bitişe (finish to finish), başlangıçtan bitişe (start to finish), başlangıçtan başlangıca (start to start) ve bitişten başlangıca (finish to start). Hoca örnek istediğinde, bir tanesine örnek vermenin zor olduğunu ama üçüne örnek verilebileceğini söyledi.

> **Ders sonrası not:** Öğrenciler ilişki tiplerini sayarken “finish to start to finish”, “start to start to start” gibi karışık ifadeler kullandı. Kastedilen büyük olasılıkla şu dört tiptir: FS, SS, FF ve SF.

Bitişten başlangıca (FS). Bir öğrencinin örneği: Testin başlaması için geliştirmenin bitmesi gerekir. Geliştirme bitmeden test başlamaz.

Başlangıçtan başlangıca (SS). Hocanın örneği analiz aşamasındandı. Bilgi toplama 1.1, modelleme 1.2 olsun. Bilgi toplamaya başlandığı andan itibaren model de oluşturulmaya başlanabilir. Hoca bunu aktivite numaralarıyla da ifade etti: Bilgi toplama diyelim ki uygulamanın başından beri beşinci aktivite, modelleme altıncı aktivitedir. Beşinci aktivitenin başlamış olması gerekir ki altıncı aktivite başlasın. Buraya bir ölçüt de konabilir: Altıncı aktivite, beşinci aktivite başladıktan bir ya da iki gün sonra başlar.

Hoca dersin ilerleyen kısmında aynı fikre üç günlük bir gecikmeyle döndü. Önce gerçekleştirilecek sistemle ilgili bilgi toplanmaya başlanır. Üç gün sonra da veri akış diyagramı ya da use case diyagramı ve belgeleri ile kavramsal sınıf diyagramları hazırlanmaya başlanır.

> **Ek bilgi (derste anlatılmadı):** FF ilişkisinde ikinci aktivite ancak ilki bitince bitebilir. SF ilişkisinde ise ikinci aktivite ancak ilki başlayınca bitebilir. SF’nin kullanım alanı dardır, bu yüzden örneği en zor olan tip genellikle odur. Hocanın “bir tanesine zor veririz” sözüyle kastettiği büyük olasılıkla SF’dir.

Bağlantı düzeldikten sonra bir öğrenci WBS düzeyleriyle ilgili bir soru sordu.

Soru: WBS’te 1 numaralı kısım analiz, 1.1 de “SRS’in yazılması ve tamamlanması” olsun. 1.1 birinci seviye mi, ikinci seviye mi?

Cevap: İkinci seviye. Bütün alt ayrıntılar kapatılırsa elimizde dört aşamalı bir plan kalır: analiz, tasarım, gerçekleştirme, test. İkinci düzey ayrıntılar açılırsa 1.1, 1.2 gibi başlıklar görülür.

8. İş Kırılım Yapısı Örnekleri: Şelale ve Sarmal Model

Hoca, WBS örneğini öğrencilerle birlikte yazıya döktü: önce şelale model, sonra sarmal (spiral) model.

8.1 Şelale Model

Şelale modelde bir adım bittiğinde diğer adım başlar. İkinci düzeye kadar yazılan örnek şöyledir:

```
1.0 Analiz
    1.1 Bilgi toplama
    1.2 Modelleme
    1.3 SRS hazırlama
2.0 Tasarım
    2.1 Mimari tasarım
    2.2 Ayrıntılı tasarım
3.0 Gerçekleme
    3.1 Kodlama
    3.2 Veri tabanının yaratılması
    3.3 Veri tabanına ilk bilgilerin girilmesi
4.0 Test
    4.1 Bütünlük testi
    4.2 Sistem testi
    4.3 Kullanıcı testi
```

Bu örnekte analiz aşamasında ön inceleme ayrı bir madde değildir. Hocanın birkaç dakika önce sözlü verdiği örnekle numaraların farklı olmasının nedeni budur. Örneğe üçüncü düzey de eklenebilir ya da aynı çalışma başka bir model için yapılabilir.

8.2 Sarmal Model

Hoca sonra aynı soruyu sarmal model için sordu: WBS'te hangi temel başlıklar olacak? Yanıt modelin ana fikrinden çıkar. Bu ana fikrin iki parçası vardır:

- Süreç sürekli yinelemelidir.
- Her yinelemede aynı adımlar gerçekleştirilir.

Bu nedenle genel adımlar analiz, tasarım, gerçekleme ve test diye ayrılmaz. Üst düzeyi evreler oluşturur: prototip oluşturma, ürün oluşturma, ürünü zenginleştirme ve bakım. Her evrede altı adım mutlaka yapılır: o evre için plan oluşturmak, müşteriden bilgi almak, risk analizi yapmak, tasarımı gerçekleştirmek, yapım (construction) aşamasını yürütmek ve müşterinin değerlendirmesine sunmak.

```
1.0 Prototip oluşturma
    1.1 Plan hazırlama
    1.2 Müşteriden bilgi alma
    1.3 Risk analizi yapma
    1.4 Analiz ve tasarım
    1.5 Yapım: ayrıntılı tasarım, kodlama ve mühendislik testi
    1.6 Müşteri değerlendirmesine sunma
2.0 Ürün oluşturma         (2.1-2.6: aynı altı adım)
3.0 Ürün zenginleştirme    (3.1-3.6: aynı altı adım)
4.0 Bakım                  (4.1-4.6: aynı altı adım)
```

Hoca ilk evrenin adımlarını yazdıktan sonra bunları kopyaladı ve yalnızca numaralarını değiştirdi. Prototip oluşturma evresinde bile plan, risk analizi ve müşteri değerlendirmesi vardır.

8.3 WBS Neden Önemli?

WBS oluşturulduğunda adımlar daha kolay takip edilir. Her adımın, projenin büyüklüğüne benzer projelerde ortalama ne kadar sürdüğü izlenmeye çalışılır. Her evredeki her adım için ne kadar bütçe gerektiğini bulmak amacıyla önce ne kadar iş gücü gerektiği bulunur. Sonra toplam bütçe hazırlanır. Ama bunların hepsi, önce kullanılan sürece uygun bir WBS kurulmasına bağlıdır.

> **Hocanın vurgusu:** WBS, kullanılan yazılım geliştirme süreç modeline uygun olmalıdır. Şelale ve sarmal model için görülen yapıyı RAD gibi başka modeller için de geliştirebilmek gerekir.

9. Planın Devamı: İş Gücü, Görev Ataması, Bütçe ve Kalite Planı

Hoca WBS'ten sonra plana döndü ve gelinen noktayı özetledi: Kapsam oluşturuldu, süreç modeli seçildi, risk analizi yapıldı, varsayım ve kısıtlar belirlendi, alt aktiviteler çıkarıldı. Zaman çizelgesinde aktiviteler Gantt diyagramıyla gösterilir, aralarındaki ilişkiler de eklenir.

9.1 Süre ve İş Gücü

Süre tahmini önceki projelerden gelen geri bildirime dayanır. Örneğin önceki projelerden alınan bilgiye göre bilgi toplama aşaması bir hafta ya da 15 gün sürecektir. Aktiviteler arası ilişkiler de belirlendiği yerlere konarak çizelge tamamlanır.

Zaman çizelgesi, aktivite ilişkileri ve PERT diyagramı yardımıyla, bu işleri gerçekleştirmek için ne kadar iş gücü gerektiği eski projelerle karşılaştırılarak bulunur. İş gücü yazılırken tipi de belirtilir. Hocanın örneği: 6 kişi-ay sistem analisti iş gücü ve 1 kişi-ay proje yöneticisi iş gücü gerekiyor.

9.2 Görev Ataması ve Bütçe

İş gücünden sonra görev ataması gelir. Görev paylaşımı yapılır ve bunun sonucuna göre bütçe hazırlanır. Bütçe için bakılan sorular şunlardır: Ne kadar süre gerekiyor, hangi kişi ya da hangi görevdeki kişi atandı?

9.3 Kalite Planı ve Kilometre Taşları

Ardından proje kalite planı gelir. Burada projede hangi aralıklarla kilometre taşı (milestone) oluşacağı belirtilir.

Soru: Kilometre taşı nedir?

Cevap (bir öğrenci, hoca tamamladı): Projede gelinen önemli noktalardır.

Kalite planında önce kilometre taşları belirtilir. Projenin dışarıdan bir sponsoru varsa sponsor, belli dönüm noktalarına gelinmesini beklemeden değerlendirme yapmak isteyebilir. Örneğin kalite planı buna göre oluşturulur ve altı ayda bir değerlendirme yapılır.

Değerlendirmede planlanan ile gerçekleşen arasındaki farklar ele alınır:

- Planlanan zaman ile gerçekleşen zaman uyumlu mu?
- Planlanan iş gücü ile gerçekleşen iş gücü uyumlu mu?
- Planlanan bütçe ile gerçekleşen bütçe arasında fark var mı?

10. Yazılım Gereksinim Spesifikasyonu

Proje planına gereksinim spesifikasyonu da eklenebilir. Gereksinim analizinde belli bir noktaya gelinmişse SRS belgesi planın bir parçası yapılabilir, çünkü proje kapsamını daha ayrıntılı açıklar.

Hoca aradan sonra bu kavrama döndü. Kavramın, geçen dönem yazılım kalite dersini alanlar için daha açık olduğunu söyledi. O dersin test projelerinde kendi projeleri için gereksinim spesifikasyonu yazılmıştı. Aynı belge iki yerde işe yarar:

- **Testte:** Kullanıcı testi buradaki fonksiyonel gereksinimlere göre, sistem testi de fonksiyonel olmayan gereksinimlere göre yapılır.
- **Projeye başlarken:** Projenin kapsamı ve ayrıntıları, müşterilerin istediği ve yazılım geliştiricilerin gerçekleştirmesi beklenen fonksiyonlar bu belgede tanımlanır.

  Ders sonrası not: Hoca sistem testini fonksiyonel olmayan gereksinimlerle eşleştirdi. Bu büyük olasılıkla tekrar için yapılmış bir sadeleştirmedir. Sistem testi genel olarak fonksiyonel gereksinimleri de kapsayabilir.

Bu yüzden gereksinim spesifikasyonu proje planının tamamlayıcısıdır. Fonksiyonel gereksinimler dendiğinde işin yapısıyla ilgili gereksinimler anlaşılır. Bunlar iki biçimde verilebilir:

- Alt alta hiyerarşik maddeler olarak yazılır.
- Modellenir ve modelin sonucu aktarılır. Bu durumda use case diyagramı ya da veri akış diyagramı (data flow diagram) ile bunların belgeleri verilir.

Hoca bu kavramların yazılım mühendisliği derslerinde verildiğini, bu yüzden herkesin böyle bir belge hazırlayabileceğini düşündüğünü söyledi. Birkaç öğrenciye sordu, sorun olmadığı yanıtını aldı.

> **Hocanın vurgusu:** SRS, müşteriyle yazılım geliştirici arasında anlaşma niteliğinde bir belgedir. Bu nedenle plan dokümanlarına eklenmesi uygundur.

11. Ekip ve Organizasyon Yapıları

Görev ataması, bütçe ve ekip yapısının tanımlanması ayrı bir madde olarak da yazılabilir. Hoca plana “proje ekip yapısının belirlenmesi” maddesini ekledi. Hangi işi kimin yapacağı belirlendikten sonra projenin genel organizasyon sisteminin nasıl olacağına karar verilir. Bu konu bir önceki derste işlendiği için üzerinden hızlıca geçildi.

11.1 Ekip Yapıları

Proje ekipleri üç biçimde oluşturulabilir:

- **Demokratik:** Bütün üyeler eşittir. Daha az kişinin olduğu ekiplerde kullanılır.
- **Şeflik:** Daha az sayıda kişiden oluşan bir ekipte yönetimi bir kişi yürütür, kararları o alır ve uygular. Kararların doğruluğu ve tutarlılığı şefin yeteneğine bağlıdır.

- **Hiyerarşik:** Bir proje yöneticisine bağlı birkaç ekip oluşur. Altta görevleri ve sorumlulukları dağıtacak şefler bulunur, proje yöneticisi de bunları takip eder. Proje yöneticisi yerine takım lideri gibi farklı adlar da kullanılabilir.

Profesyonel projelerde, daha kalabalık ve karmaşık projelerde hiyerarşik sistemler daha önemlidir. Hiyerarşik yapı söz konusu olduğunda karşımıza proje organizasyon yapıları çıkar.

11.2 Organizasyon Yapıları

Fonksiyonel organizasyon. Çalışanlar uzmanlık alanlarına göre bölümlenir. Yapılan iş yazılım geliştirmedir, öyleyse bu işin uzmanlık alanları nelerdir diye sorulur. Proje yöneticisi bu alanlara göre çalışma grupları oluşturur: sistem analistleri, pazarlama ve ürünün yayılması, bakım, yazılımcı ya da programcıların bağlı olduğu ekipler gibi.

Proje organizasyonu. Sistemin ana işlevleri çıkarılır. Her modülün, her parçanın gerçekleştirilmesi için gereken tüm uzmanlık alanlarının yönetimi bu yapıda sağlanır. Hocanın örneği bir muhasebe ve mali işler uygulamasıydı. Burada stok kontrol takım lideri, finans modülü takım lideri, MIS modülü takım lideri ve müşteri ekranları takım lideri gibi ekipler kurulur.

Soru: Fonksiyonel organizasyon ile proje organizasyonu arasındaki fark nedir?

Cevap: Fonksiyonel organizasyonda ekipler, yazılım geliştirme işinin alt başlıkları ve uzmanlık alanları düşünülerek oluşturulur. Proje organizasyonunda ise projenin alanına, gerçekleştirdiği işe ve bu işin alt işlemlerine bakılır. Yani ekipler projenin modüllerine göre kurulur.

Matris organizasyon. Bu yapı raporlama ve yetki ilişkileri açısından farklıdır. Şirketin genel yönetiminde fonksiyonel bir yapı bulunduğu düşünülür: sistem analistleri, programcılar, yazılım mühendisleri ya da yazılım mimarları ve tasarımcılar için grup yöneticileri vardır, çalışanlar bu ekiplere bağlıdır. Ama projede de alt modüllere göre yaptıkları işler vardır ve bu işlerde proje yöneticilerine bağlıdırlar. Böylece yetki birden fazla kişiye dağıtılmış olur. Uzmanlar kendilerine verilen işleri hem birim yöneticilerine hem de proje yöneticilerine raporlar.

Hocanın gösterdiği şekilde yapı şöyledir: Genel müdürün altında yazılım mühendisleri lideri, sistem analistleri lideri ve programcılar lideri vardır. Bunların yanında A projesi yöneticisi ve B projesi yöneticisi bulunur. Bir projede bir sistem analisti, bir programcı, bir yazılım mühendisi ve bir test grubu çalışabilir. Bu kişiler projedeki görevleriyle proje yöneticisine, genel işleriyle birim yöneticilerine bağlıdır.

Yapı	Ekipler neye göre kurulur?	Çalışan kime bağlıdır?
Fonksiyonel	Yazılım geliştirme işinin uzmanlık alanlarına göre	Uzmanlık grubu ve onun üzerinden proje yöneticisi
Proje	Projenin modüllerine ve ana işlevlerine göre	Modülün takım lideri
Matris	Hem birimlere hem projelere göre	Hem birim yöneticisi hem proje yöneticisi

12. Proje Zamanlaması: Neden CPM Yerine PERT?

Plan içinde üzerinde durulan bir diğer konu proje zamanlamasıydı. Gantt diyagramı ile PERT ve CPM (critical path method, kritik yol yöntemi) şemaları kullanılır. Hoca uygulamalarda CPM yerine PERT kullanıldığını hatırlattı ve nedenini sordu.

Soru: Neden CPM yerine PERT kullanıyoruz?

Cevap (bir öğrenci): PERT ağında asıl proje gidişatı görülmek istenir. Kritik yolu etkileyen bir kısım var mı, kritik yol eforu azaltılarak nasıl ilerletilebilir, bunlar görülür.

Hoca cevabı beğendi ve tamamladı. Şebeke analizi, yani ağ diyagramı, projedeki aktivitelerin hangi yollar üzerinde dizileceğini gösterir. CPM de bunu aynı şekilde gösterir. Fark sürelerdedir.

CPM belirlilik hâlinde kullanılır. Yani bütün alt aktivitelerin, şelale için konuşursak analiz, tasarım, gerçekleştirme ve testin, hangi uygulama olursa olsun sürelerinin belirlenebildiği var-

sayılır. Oysa yazılım projelerinde “her projede analiz şu kadar sürer, tasarım şu kadar sürer, gerçekleştirme şu kadar iş gücüyle tamamlanır” denemez. Bu süreler belirsizdir.

PERT bu belirsizlik için önce bir tahmin yapar. Her aktivitenin en kötümser, en iyimser ve şimdiye kadarki ortalamalara göre en olası sonucu ele alınır. Böylece süreler önce hesaplanır, sonra işleme başlanır. CPM ise biraz daha basittir ve belirlilik hâlinde kullanılır.

> **Ek bilgi (derste anlatılmadı):** PERT’te üç tahminden beklenen süre genellikle $t_e = \dfrac{a + 4m + b}{6}$ ile hesaplanır. Burada a iyimser, m en olası, b kötümser süredir.

13. Yazılım Ölçümü

Hoca ekran paylaşımını değiştirip etkin proje yönetimi başlığına geçti. Proje yönetimi dendiğinde ölçüm de karşımıza çıkar. Yazılım metrikleri (software metrics) neden önemlidir? Hoca daha önce andığı klişe cümleyi hatırlattı.

> **Hocanın vurgusu:** Ölçemediğinizi yönetemezsiniz.

Daha iyi bir yönetim için önce gerçekleştirilen işleri ölçmek gerekir. Mühendislik yaklaşımı da bunu gerektirir. Yazılım projelerindeki işleri, sonuçları ve ürünü ölçülebilir hedeflere dönüştürmek için metriklerden yararlanılır. Yazılım ürününün özellikleri, kullanıcı özellikleri, bakım özellikleri ve kalite faktörleri metriklerle daha ölçülebilir hâle gelir.

Yazılımı ölçmek için önce büyüklük ölçümü yapılır. İki temel ölçüm vardır: komut satırı sayısı (LOC, line of code) ve işlev noktası (function point).

LOC doğrudan ölçümdür. Satırlar doğrudan sayılır ve bu sayı büyüklük hakkında bir fikir verir.

İşlev noktası dolaylı ölçümdür. Yazılımın farklı bilgilerini birleştirerek sonuç verir. Hem büyüklük hem karmaşıklık hakkında bilgi verir ve ilk olarak ticari uygulamalar için başlatılmıştır. Hesap için kullanıcının yazılımla etkileşimine ilişkin şu sayılar kullanılır:

- Girdi sayısı.
- Çıktı sayısı.
- Sorgulama sayısı.
- Dosya sayısı.
- Dış arabirim sayısı.

Hesap iki adımdır. İlk adımda ham değer bulunur: Her satır için sayı, ilgili ağırlık faktörüyle çarpılır. Ağırlık faktörü, sayılan öğenin basit, ortalama ya da karmaşık sınıfına girmesine göre seçilir. Sonra bunlar toplanır.

İkinci adımda bu toplam doğrudan kullanılmaz, bir düzeltme faktörüyle düzeltilir:

$$FP = \text{sayı toplamı} \times \left(0{,}65 + 0{,}01 \sum F_i\right)$$

Düzeltme faktörü için 14 soru vardır. Hoca bunlardan bazılarını saydı: Yedekleme gerekli mi? İletişim altyapısı gerekli mi? Performans kritik mi? Çevrim içi veri girişi var mı? Her soru sıfırdan beşe kadar bir önem değeri alır. Beş, o özelliğin gerçekten çok önemli olduğu anlamına gelir. Bu değerler toplanır ve formüldeki $\sum F_i$ yerine konur.

> **Ek bilgi (derste anlatılmadı):** 14 soru 0-5 arası puanlandığı için $\sum F_i$ en az 0, en çok 70 olur. Böylece düzeltme çarpanı 0,65 ile 1,35 arasında kalır.

14. Maliyet Tahmin Yöntemleri

Hoca maliyet tahmin yöntemlerini dört başlıkta hatırlattı: bilirkişi takdiri, Delphi yöntemi, analiz yöntemi ve istatistik yöntemler. İstatistik modeller, eski projelere dayanarak daha bilimsel tahminler yapmayı sağlar.

Bilirkişi takdiri ve Delphi. Delphi, bilirkişi takdirini genişleten bir yöntemdir. Hoca bunun birçok yerde kullanılan bir çoklu karar verme yöntemi olduğunu söyledi.

Analiz yöntemi. Ürün, geliştirme işleminin geçtiği aşamalara ya da alt modüllerine ayrılır. Her parçanın maliyet tahmini ayrı ayrı yapılır ve sonra birleştirilir. Ekip yapısındaki ayrıma benzer bir mantık burada da kullanılır.

Hallows'un kitabında bu birleştirmeye sonradan "birleştirme şaşkınlığı" adı verilmiştir. Her modüle yeterince zaman verilip süreler toplandığında, projenin tamamlanma süresi istenen bitiş süresinden çok daha uzun çıkar. Şaşkınlık buradan gelir. Bu yüzden tahminin üzerinden birkaç kez geçilir:

1. Her alt modül ya da alt iş için ne kadar süre, maliyet ve iş gücü gerektiği hesaplanır.
2. Sonuçlar yukarı doğru birleştirilir.
3. Toplam çok büyük çıkarsa aşağı doğru yeniden gidilir ve işler daha kısa sürede yapılacak biçimde gözden geçirilir.

İstatistik modeller. Bu modellerde de analiz yöntemi kullanılır ya da alt işlemlere göre tahmin yapılır. Hoca şu örnekleri verdi:

- **KLOC kullananlar:** COCOMO ve Putnam modeli.
- **İşlev noktası kullananlar:** Albrecht yöntemi gibi maliyet tahmin yöntemleri.

Bu modeller arasında formülün girdisi değişir. Temel COCOMO'da iş gücü, KLOC'un bağımsız değişken olarak kullanılmasıyla elde edilir. Süre (T) ise iş gücünün ne kadar süreye yayılmasının doğru olacağı düşüncesiyle bulunur. COCOMO'nun temel, orta ve daha gelişmiş (karmaşık) biçimleri vardır.

> **Ek bilgi (derste anlatılmadı):** Temel COCOMO'nun genel biçimi $E = a \cdot (KLOC)^b$ ve $T = c \cdot E^d$ şeklindedir. Burada E iş gücü, T süredir. Katsayılar proje tipine göre değişir.
>
> **Ders sonrası not:** Altyazıdaki "DELF/DERF", "Kokomo", "Carlock/K-Log" büyük olasılıkla Delphi, COCOMO ve KLOC'tur. "Putnam-Reilly" ise büyük olasılıkla Putnam modelinin dayandığı Rayleigh eğrisine işaret eden "Putnam-Rayleigh" adıdır.

15. Risk Analizi ve Önlemler

Hoca risk kategorilerini ayrıntılı işlendikleri için tekrar etmedi. Yalnızca o gün eklediği noktayı yineledi: Varsayımlar ve kısıtlar için de risk kategorilerine benzer kategorilerle tanımlamalar yapılabilir.

Proje risk analizinde amaç risk tablosunu oluşturmaktır. Tablo şu sırayla kurulur:

1. Her risk kategorisi düşünülür ve o kategorideki riskler belirlenir.
2. Her riskin olma olasılığı tahmin edilir.
3. Risk gerçekleşirse etkisinin, yani yıkımının ne olacağı tahmin edilir. Böylece her riskin önemi ortaya çıkar.
4. Riskler önem sayısına göre sıralanır.
5. Belli bir miktarın altında kalanlar takip edilmez, önemli çıkanlar takip edilir.
6. Her risk için önlem belirlenir.

Hoca iki örnek verdi. İlki, kilit proje ekip elemanının işten ayrılması riskidir. Bunun için yedek bir eleman işe alınabilir, kişiyle sözleşme imzalanabilir ya da ücreti artırılabilir. Bu önlemlerin de maliyeti vardır ve takip edilirler. Olasılık artmışsa önlemler de ona göre devreye alınır.

İkinci örnek, yazılım geliştirilirken kullanılacak bir donanım ürününün siparişinin gecikmesidir. Bu da bir risktir. Her zaman ortaya çıkmaz, ama çıktığında ne yapılacağı bilinmeli, çıkma olasılığı da takip edilmelidir. Hoca bütün bu önlemlerin önceden yapılan çalışmalarla belirlendiğini söyledi.

16. Planın Çalıştırılması: Hazırlık, Planlama ve İzleme

Hoca dersin sonunda bir soru üzerine, uygulama projesi bağlamında yol haritasının son adımını somutlaştırdı. Önemli olan, seçilen konuda proje planını sıfırdan kendi başına yapmak ve projeyi en az gereksinim analizine kadar çalıştırmaktır. Bu yüzden proje planının altına gereksinim analizini de yazdığını söyledi.

Böyle bir çalışma üç evreyi kapsar: projenin hazırlık aşaması, planlama aşaması ve çalıştırılması aşaması.

> **Hocanın vurgusu:** Çalıştırma, projeyi takip edip ilerlerken planlanan ile gerçekleşen arasındaki farkları izlemek ve önlem almaktır.

Dersin Toparlaması

Bu tekrar dersi, proje yönetiminin yalnızca bir plan belgesi hazırlamak olmadığı fikri üzerine kuruldu. Yol haritası projenin anlaşılması ve tanımlanmasıyla başlar, planın oluşturulmasıyla sürer, planın çalıştırılması ve takibiyle devam eder. Plan ise kapsamdan SRS'e uzanan maddelerden oluşur. Bu maddelerin ortasında duran WBS, seçilen süreç modeline uygun kurulmalıdır. Şelale modelde üst başlıklar analiz, tasarım, gerçekleme ve testtir. Sarmal modelde ise evreler vardır ve her evrede aynı altı adım yinelenir.

Dersin ikinci yarısında planın dayandığı konular hatırlatıldı: ekip ve organizasyon yapıları, belirsizlik nedeniyle PERT'in tercih edilmesi, LOC ve işlev noktasıyla ölçüm, maliyet tahmin yöntemleri, risk tablosu ve önlemler. Bu konuları birbirine bağlayan düşünce, planlananı ölçülebilir kılmak ve gerçekleşenle karşılaştırmaktır.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.